# Sre Cibi Varsshan K

**C++ / GPU / Graphics / Game Programmer**

srecibi@gmail.com | github.com/Orion2726 | FPS Controller Demo

## SUMMARY

Final-year Computer Science student and C++ programmer building performance-oriented software, GPU compute pipelines, and real-time rendering systems, alongside gameplay programming in C# and Unity. Hands-on experience with GPU compute, real-time rendering, shader programming, physics-based movement, modular game systems, parallel computation, and graphics programming. Seeking C++/GPU/Graphics Programmer or Unity Game Programmer roles in performance engineering, rendering, or gameplay systems.

## EDUCATION

**VIT Chennai** Expected 2027

B.Tech in Computer Science

Relevant coursework: Computer Graphics, Data Structures & Algorithms, Machine Learning, Computer Vision

## PROJECTS

**GPU-Aided Bulk Image Encryption (ChaCha20)** *C++, OpenGL, GLSL*

- Implemented ChaCha20 image encryption as an OpenGL compute pipeline, mapping ARX-based keystream generation across GPU workgroups.
- Designed Shader Storage Buffer Object (SSBO)-based buffer handling for batch image processing and metadata-driven reconstruction for lossless decryption.
- Applied parallel computation and GPU buffer management to move encryption work from CPU-side iteration to GPU execution.

**GPU Ray Marcher** *C++, OpenGL, GLSL*

- Built a real-time ray marching renderer driven entirely through fragment shaders, evaluating Signed Distance Functions (SDFs) for spheres, boxes, and planes with boolean composition (union, subtraction, intersection).
- Implemented an FPS-style free-fly camera within the engine for real-time scene navigation and debugging.
- Built a multi-pass rendering pipeline using Framebuffer Objects (FBOs), applying Sobel edge detection as a post-processing pass on the final rendered output.

**CPU Ray Tracer** *C++*

- Built a CPU ray tracer in C++ implementing a positionable camera, anti-aliasing via multi-sample pixel averaging, and recursive ray-color evaluation with configurable bounce depth.
- Implemented diffuse (Lambertian), metal (adjustable fuzz), and dielectric (refractive, Schlick-approximation Fresnel) materials for physically based light transport.
- Added depth-of-field through a defocus-disk camera model with adjustable aperture and focus distance, and measured render performance with built-in timing instrumentation.

**Deep Q-Network Space Shooter** *Python, PyTorch*

- Built a pixel-based Dueling Double DQN agent that learns to play a custom Pygame space-shooter directly from raw screen pixels (4 stacked grayscale 84x84 frames), with no hand-crafted state features.
- Implemented a CNN feature extractor with separate value/advantage streams (Dueling architecture) and a Double DQN target network to reduce Q-value overestimation.
- Stabilized training with a 50,000-transition replay buffer, Huber loss, gradient clipping, epsilon-greedy exploration decay, and wave-based reward shaping across multiple enemy types and attack patterns.

**Doofus Diary** *Unity 6, C#*

- Built a 3D platform-survival game with modular gameplay systems, dynamic platforms, progressive difficulty, scoring, UI, audio, and player progression.
- Implemented single- and double-jump progression, Rigidbody-based movement, and a spring-damper camera with momentum-aware following.
- Structured player movement, platform spawning, game state, scoring, UI, audio, and JSON-based configuration as focused gameplay systems; shipped a playable WebGL build.

**Physics-Based FPS Controller & Rendering Sandbox** *Unity, C#*

- Built a physics-oriented FPS controller with custom acceleration, friction, air control, jumping, and momentum-based traversal.
- Implemented Quake-style movement features including coyote time and jump buffering, with runtime physics and responsive player movement as primary design constraints.
- Used the project as a real-time rendering sandbox for shader-driven visual effects and lighting.

**Slot Machine** *Unity 6, C#, ShaderLab, HLSL*

- Built a 3-reel slot-machine game with weighted random number generation, staggered reel animation, pixel-perfect alignment, and data-driven symbol configuration.
- Developed a custom URP post-processing shader combining palette quantization, ordered dithering, and Sobel edge detection.
- Shipped a playable WebGL build combining gameplay logic, animation, UI, and custom GPU shader effects.

## TECHNICAL SKILLS

**Programming Languages:** C++, C, C#, Python, Bash
**GPU & Graphics Programming:** OpenGL, GLSL, Compute Shaders, Shader Storage Buffer Objects (SSBO), HLSL, ShaderLab, Ray Tracing, Ray Marching, Real-Time Rendering
**Game Development:** Unity 6, Gameplay Systems, Rigidbody Physics, Physics-Based Movement, Game AI, UI Systems, Data-Driven Configuration, WebGL
**Algorithms & Systems:** Data Structures & Algorithms, Parallel Computing, Memory Management, Buffer Management
**Machine Learning:** PyTorch, Convolutional Neural Networks (CNN), Deep Q-Networks (DQN), Reinforcement Learning, Pygame
**Tools & Platforms:** Linux, Git, Version Control